

Graph Visualization Specification Index

Normative documentation map for implemented G1-G5

mdstats

Contents

Purpose and status	1
AI context summary	2
Specification map	3
Dependency direction	5
Public API by module	5
Common graph-view core	5
Styling	5
Two-dimensional renderer	6
Atomic-connectivity adapter	6
Framework-topology adapter	6
Periodic graph display	6
Interactive three-dimensional renderer	6
Responsibility boundaries	6
Scientific modules	6
Visualization core	6
Periodic preparation	7
Styling	7
Renderers	7
Scientific adapters	7
Cross-module identity rules	7
Source scientific identity	7
Display identity	7
Renderer identity	7
Filtering and periodic order	7
Current implementation status	7
Implemented G1-G3	7
Implemented G4	8
Implemented G5	8
Na-LTA system-integration contract	8
Shared input constraints	8
Optional dependency policy	8
Future compatibility	9
Normative language	9
Supersession and migration	9
Acceptance checklist	9
LD8-P0 and LD9-V0 density optimization specifications	9
Interactive density mesh module specifications	10

Purpose and status

This index defines the modular documentation boundary for the mdstats graph-visualization subsystem.

Current status:

G1-G3 implemented and validated in mdstats 0.10.0
G4-G5 implemented and validated in mdstats 0.11.0
G6 framework-topology adapter implemented in mdstats 0.13.0

Style compact dots and edge-only node display implemented in mdstats 0.13.1
Plot-D1/D2 registered framework and trajectories implemented in mdstats 0.19.30a0
Plot-D3 atomic density implemented in mdstats 0.19.31a0
LD0-R1 through LD8-S4 and LD9-V0 through LD9-V4 implemented through mdstats 0.19.62a0
LD16 Stage 1 regression locks implemented in mdstats 0.19.74a0
LD17 Stage 2 explicit mesh-face contracts implemented in mdstats 0.19.75a0
G7-G8 architectural placeholders only

The former monolithic G1-G3 document `graph_visualization_core_atomic_adapter_specification_v1.*` remains historical but is not a normative API owner.

AI context summary

When using these documents as implementation context, preserve the following invariants:

1. The scientific graph remains authoritative.
2. `DecoratedGraphView` is immutable renderer input, not a scientific result.
3. Stable scientific keys are distinct from dense source positions and display keys.
4. Filtering, periodic materialization, styling, layout, and rendering never alter scientific graph identity.
5. Omitted nodes and edges are explicit; silent sampling is forbidden.
6. Periodic replicas and ghosts are display objects only.
7. `PeriodicGraphView` is shared by 2-D and 3-D renderers.
8. Renderers do not infer bonds, framework topology, rings, sites, or cages.
9. Atomic canonical image shifts remain distinct from frame-local display shifts.
10. Plotly remains optional and lazily imported.
11. Node-display modes alter artists only; hidden nodes remain in scientific and display mappings.
12. Framework and future ring adapters consume their scientific modules rather than reimplementing them.
13. Projected framework shifts remain distinct from frame-local display shifts.
14. One concept has exactly one normative specification owner.

The full separation is

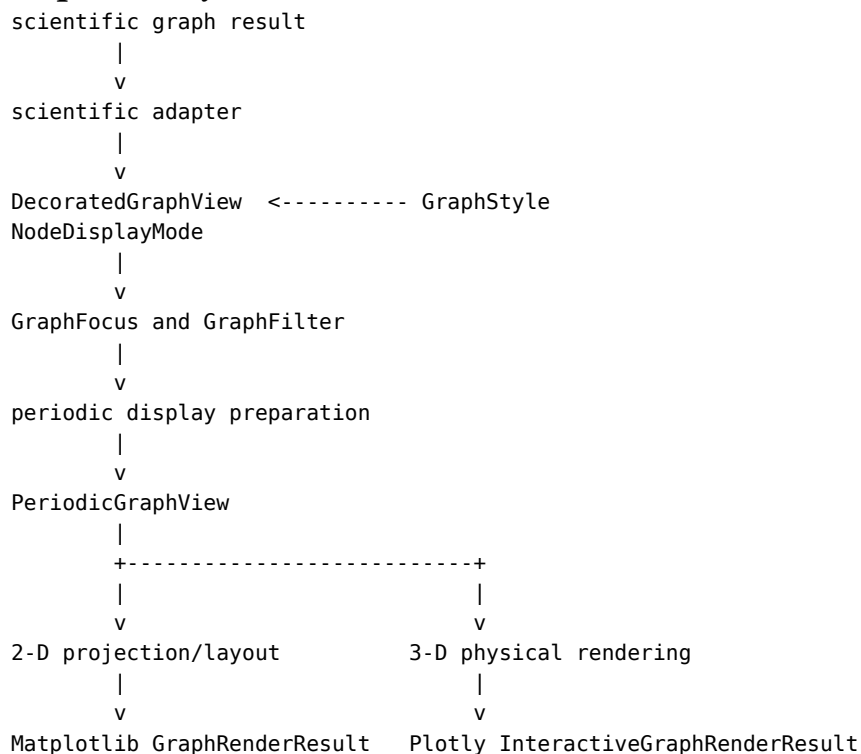
scientific result $\rightarrow$ `DecoratedGraphView` $\rightarrow$ source selection $\rightarrow$ `PeriodicGraphView` $\rightarrow$ renderer output .

Specification map

Normative document	Status	Source ownership	Primary public API
graph-view core (PDF)	implemented	graph_view.py, graph_errors.py	graph data, focus, filters, complexity
graph style (PDF)	implemented	graph_styles.py	palettes, styles, rules, labels
2-D renderer (PDF)	implemented; G4 integra- tion complete	graph_2d.py	physical/schematic 2-D lay- out and Matplotlib output
atomic-connectivity adapter (PDF)	implemented with 2-D and 3-D wrappers	atomic_connectivity_graph.py	atomic state/transition graph views and wrappers
framework-topology adapter (PDF)	implemented G6	framework_topology_graph.py	projected framework and retained atomic-path views
framework-dynamics over- lays (PDF)	implemented Plot-D1/D2	framework_dynamics.py	registered mean framework and selected atomic trajec- tories
periodic atomic density (PDF)	implemented Plot-D3	atomic_density.py, framework_dynamics.py	normalized occupancy fields and browser-stable probability-mass meshes
LD2-A sparse density clouds (PDF)	implemented	density_node_cloud.py, framework_dynamics.py	backend-neutral HDR de- tails and deterministic logi- cal-node cloud rendering
LD2-B sparse density meshes (PDF)	implemented	density_sparse_mesh.py, framework_dynamics.py	periodic candidate-cell meshing, clipping, seam val- idation, and winding fall- back
LD3 sparse framework den- sity (PDF)	implemented	framework_density.py, framework_dynamics.py	sparse framework-vertex occupancy and framework- edge arc-length channels
LD4 automatic backend se- lection (PDF)	implemented	density_backend_selection.py, framework_dynamics.py	exact transactional dense- versus-local-sparse selec- tion
LD11 default automatic den- sity policy (PDF)	implemented	density_contracts.py, atomic_density.py, framework_density.py, framework_dynamics.py	canonical periodized opera- tor and physical-resolution- first automatic backend de- faults
LD5 sparse optimization and caching (PDF)	implemented	density_sparse_optimization.py, density_preparation/plan- ning modules	optimized/reference sparse evaluation and bounded canonical-support caching
density mesh simplification (PDF)	implemented	density_mesh_simplify.py	periodic QEM reduction with seam, topology, and scalar-field fidelity checks
density scene fitting (PDF)	implemented	density_scene_fit.py	closed-loop simplification, recontouring, and exact profile compliance
density mesh contracts (PDF)	implemented	density_mesh_contracts.py	raw extraction, visual tar- get, and standalone final face semantics
browser-ready data (PDF)	implemented	density_renderer_hub.py	front-most application, raw scene rendering, and profile validation

The Graph Visualization Architecture Manual remains the high-level owner of goals, stages, dependency policy, and future scientific adapters. It does not redefine public dataclass fields.

Dependency direction



Dependencies flow downward. The common graph-view module imports no scientific adapter or renderer. G4 may use the private source-selection implementation owned by the core, but it does not redefine focus or filtering. G5 consumes G4 and never performs independent periodic replication.

Public API by module

Common graph-view core

DecoratedGraphView
 GraphFocus
 AttributeSelection
 GraphFilter
 GraphComplexityPolicy
 GraphComplexityReport

The shared public exception hierarchy also originates in this layer.

Styling

ChemicalColorPalette
 NodeStyle
 NodeStylePatch
 EdgeStyle
 EdgeStylePatch
 NodeStyleRule
 EdgeStyleRule
 GraphLabelOptions
 GraphStyle

Two-dimensional renderer

GraphLayoutOptions
Graph2DRenderOptions
GraphRenderResult
plot_decorated_graph_2d

Atomic-connectivity adapter

Implemented:

graph_view_from_atomic_connectivity
graph_view_from_connectivity_transition
plot_atomic_connectivity_2d
plot_connectivity_transition_2d

Implemented in G5:

plot_atomic_connectivity_3d
plot_connectivity_transition_3d

Framework-topology adapter

FrameworkGraphDisplayMode
FrameworkPathSegmentKey
graph_view_from_framework_topology
plot_framework_topology_2d
plot_framework_topology_3d

The projected view preserves authoritative FrameworkEdgeKey identity. The atomic-path diagnostic view expands only the paths stored by FrameworkTopology.

Periodic graph display

PeriodicDisplayMode
PeriodicNodeRole
PeriodicEdgeRole
PeriodicNodeKey
PeriodicEdgeKey
CanonicalCellDisplay
LocalUnwrappedDisplay
ExpandedCellDisplay
PeriodicDisplayOptions
PeriodicGraphView
prepare_periodic_graph_view

Interactive three-dimensional renderer

Graph3DRenderOptions
InteractiveGraphRenderResult
plot_decorated_graph_3d

Responsibility boundaries

Scientific modules

Scientific modules own graph identity and physical meaning. Atomic connectivity owns atomic graph states and transitions. Future framework and ring modules own projection, linker contraction, primitive rings, ring geometry, sites, and cages.

Visualization core

The core owns immutable graph-view validation, stable keys, source selection, omission accounting, complexity limits, and common exceptions.

Periodic preparation

G4 owns display-node replication, image assignment, winding ghosts, expanded cell ranges, and source-to-display mappings. It performs no scientific connectivity test.

Styling

The style module owns declarative visual encodings. Style never changes source or display graph content.

Renderers

Renderers own layout coordinates or physical scene assembly, artists or traces, labels, legends, camera, and export behavior. They do not create scientific edges.

Scientific adapters

Adapters translate scientific results into `DecoratedGraphView` and attach scientific metadata. They may reconstruct frame-consistent quotient translations but must not render or replicate periodic images.

Cross-module identity rules

Source scientific identity

Stable source node and edge keys identify scientific graph objects.

Display identity

G4 display identities are

$$(k_i, \mathbf{q}_i)$$

for nodes and

$$(k_e, \mathbf{q}_s, \mathbf{q}_t)$$

for edges.

Display keys may multiply one source key across replicas. They never replace source identity.

Renderer identity

Matplotlib artists and Plotly traces are backend objects. Their insertion order is not scientific identity. Render results must preserve explicit mappings back to display and source keys.

Filtering and periodic order

The shared order is normative:

1. source focus
2. source filters
3. source omission record
4. periodic display count estimate
5. complexity enforcement
6. periodic materialization
7. style resolution
8. renderer construction

A renderer must not replicate first and then apply source-key filters, because that would make filter semantics image-dependent.

Current implementation status

Implemented G1-G3

mdstats 0.10.0 provides:

- immutable decorated graph views;
- deterministic focus and filters;

- style rules and chemical palettes;
- Matplotlib 2-D physical and schematic layouts;
- atomic connectivity state and transition adapters;
- minimal renderer-local periodic unwrapping and ghost endpoints;
- Na-LTA integration fixtures and rendered diagnostics.

Implemented G4

G4 implements generalized periodic display preparation into `periodic_graph.py` with:

- canonical-cell materialization;
- local unwrapping with residual winding ghosts;
- expanded rectangular cell ranges;
- source-to-display mappings;
- shared periodic provenance for both renderers.

Implemented G5

G5 provides an optional Plotly backend with:

- interactive physical 3-D rendering;
- hover inspection;
- perspective and orthographic cameras;
- unit-cell wireframes;
- HTML export;
- atomic state and transition wrappers.

Na-LTA system-integration contract

The relaxed Na-LTA fixture remains the primary visualization integration case. Authoritative framework counts are:

```
144 framework atoms
192 T-O edges
48 T atoms with degree 4
96 framework O atoms with degree 2
```

G4-G5 acceptance views include:

1. canonical-cell framework view;
2. local unwrapped Si-centered neighborhood;
3. expanded $2 \times 2 \times 1$ framework view;
4. framework plus illustrative Na-O contacts;
5. perspective and orthographic 3-D variants.

Replicas and ghosts may increase display counts. Every display object must map exactly to one authoritative source object.

Shared input constraints

All arrays must have deterministic ordering, finite numeric values, and shapes consistent with associated node or edge counts. Stable keys must be hashable and unique within their layer. Metadata columns must be immutable scalar sequences or one-dimensional arrays of the correct length.

Periodic preparation requires finite 3-D positions, a finite nonsingular cell, PBC flags, and frame-consistent quotient edge translations. Nonperiodic axes may not carry image translation.

No renderer may silently drop objects because of complexity. A rejecting policy must raise; `warn_and_render` must record the exceeded limits.

Optional dependency policy

The common graph view, styles, 2-D renderer, adapters, and periodic preparation must not require Plotly.

Only `graph_3d.py` may import Plotly, and it must do so lazily. Packaging should expose an optional extra such as

mdstats[interactive]

for the 3-D backend.

Future compatibility

The following planned adapters fit downstream of the same contracts:

framework_topology_graph.py

decorated projected framework vertices and linker-path edges

ring_graph.py

ring incidence, ring adjacency, ring-center and normal metadata

site_and_cage_graph.py

site, cage, and transport graphs with real-space centers

Future adapters must not add domain-specific fields to generic renderer options.

Normative language

The words **must**, **must not**, **should**, **should not**, and **may** have their usual specification meanings.

For implemented G1-G5 behavior, source and tests currently take precedence over a document discrepancy, and the discrepancy is a defect. Specifications must be updated deliberately whenever implementation review exposes a necessary change.

Supersession and migration

The modular specification set supersedes the monolithic G1-G3 document.

G4 additionally supersedes renderer-local ownership of generalized periodic graph materialization. The 2-D renderer may preserve legacy 0.10.0 options as compatibility syntax, but periodic algorithms and source-to-display semantics belong to the G4 specification.

G5 does not supersede the 2-D renderer. Both backends share graph, style, and periodic contracts while optimizing different use cases.

Acceptance checklist

The G4-G5 implementation is accepted when:

- every public field and function has one normative owner;
- G4 options cannot express invalid cross-mode combinations;
- source and display identities are mathematically explicit;
- winding and expanded-cell algorithms are deterministic;
- G5 imports Plotly lazily;
- unit-cell geometry supports triclinic cells;
- hover and trace mappings preserve source keys;
- complexity and unsupported features have explicit policies;
- atomic 3-D wrappers reuse existing adapters;
- Na-LTA interactive acceptance cases are fixed;
- Markdown and PDF versions have heading and equation parity.
- LD6 multilevel density research gate

LD8-P0 and LD9-V0 density optimization specifications

- LD8-P0 production-cutoff evidence

- LD9-V0 hard browser render budget
- LD9-V0 mesh fidelity and topology validation
- LD8-S0 periodic kernel block routing
- LD8-S1 exact periodic support atlas
- LD8-S0 packed scientific field contract
- LD8-S2 canonical target-owned direct realization
- LD8-S3 hybrid tiled direct/FFT realization
- LD12 hybrid-aware scene admission
- LD8-S4 production dispatch and downstream reuse
- LD9-V1 bounded tiled contour extraction

Interactive density mesh module specifications

- Density mesh face contracts
- Exact browser mesh budget
- Scene-wide shell allocation
- Closed-loop density scene fitting
- Periodic density-mesh simplification
- Bounded mesh-worker execution
- Browser functional and production acceptance
- Framework dynamics and partitioned topology rendering

The former Stage-1, Stage-2, combined Stages-2–9, and LD9-V0/V2/V3/V4 chronological documents have been absorbed into these module-owned specifications.